Data Structures and Algorithms

Prathibha K S
Department of Computer Science
Amrita Vishwa Vidyapeetham

Objectives

- Data Structure
- Algorithm
- Array
- Stack

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Data Structure

- Data Structure is a way of organizing data so that it can be easily retrieved and used by the program.
- Example: Array, stack, queue, linked list
- It should be designed and implemented in such a way that it reduces the complexity and increases the efficiency.

Algorithm

- An algorithm is a definite sequence of unambiguous steps to solve a computational problem. It takes an input, apply finites number of computations on it and give back the result as output.

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Example 1

Algorithm 1: Search a value *key* from a list of n numbers

Input: List, *lst* of n numbers, a value to be searched, *key*
Output: True if *key* is in list, False otherwise

```
1 i ← 1
2 while i ≤ n do
3     if lst[i] = key then
4         return True
5     else
6     i ← i + 1
7 return False
```

Program

- Program – implementation of an algorithm in a specific language.

- Program = Algorithms + Data Structure

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Abstract Data Types (ADT)

- ADT is a mathematical model that specifies the properties of a type.
- It consists of
  - Specification of the possible values of the data types.
  - Specification of the operations that can be performed on the values in terms of the operations.

  This specification is independent of implementation of the type in any language.

Abstract Data Types (ADT) - Example

Array as an ADT

An array stores a set of items which are accessed using index.

Operations in an array ADT are :-

- put(elt,indx) → stores an element , *elt* at the index *indx*.
- get(indx)→retrieves an element from the index *indx*.
- Traverse()→ visit each element of the array starting from beginning till the last element..

Examples of Data Structures

- Array
- Stack
- Queue
- Linked List
- Graph
- Tress

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Data Structures and Algorithms

Geetha Lekshmy V
Department of Computer Science
Amrita Vishwa Vidyapeetham

Objectives

Array

Array –Operations

1D Array – Definition in C, Java, Python

Array

- Sequence of elements that are numbered.
- The element number is called as an **index**.
- Each element is accessed using the index, which is an integer.
- Many array implementations(C, C++, Java, Python) - index starts with zero.
- One dimensional array implementation in C, Java, Python – array elements are stored in contiguous memory locations .
- Array stores identically typed entities.

Array - Operations

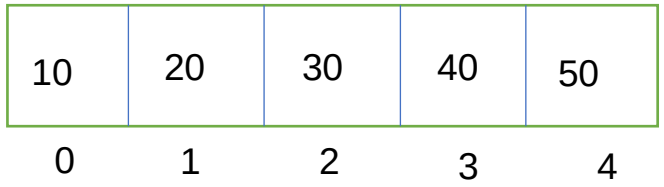
- An array is a **random access** structure.
- Two operations on an array – Store and Retrieve.
- Store puts an element to array.
- Retrieve takes an element from array.
- Both operations take constant time for execution.

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Array – Operations

- int num[] = new int[5]; → Defines an array named *num*, of **size 5** in Java language. *num* is a reference to the array object.
  - num[0] → accesses first element. (In Java index starts with 0)
  - num[1] = 10 ; → stores 10 into array, at index =1.
  - num[i] → accesses i^{th} element.
  - System.out.println(num[4]);→ prints last element.

  (Note:- last index is **size-1**)
- Logical representation of num →



- num[5] →throws arrayindexoutofbound exeception.

Definition of an 1D Array

One dimensional array – accessed using only 1 index

```
int num[5];                                  // “C” langauge

float arr[]=new float[5];                    //  Java


import numpy as np                           //Python
arr = np.array([1, 2, 3, 4, 5])
```

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Array – Declaration, Assignment

- int arr1[]; //array declaration, no memory is allocated to store array, arr1 is a reference which is null .
- arr1 = new int[10]; //arr1 refers to an array of 10 integers
- int arr2[] = new int[10]; // defines an array of 10 integers
- int arr2[] ={1,2,3,4}; //creates and initialises an array
- for(int j=0;j<**arr2.length**;j++) //**arr2.length** return size of arr1
  System.out.println(arr2[j]); //print elements in the array

Summary

- linear Data structure.
- equally sized elements.
- random access using index.
- operations are Store and retrieve.

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

References

Fundamentals of Data Structures by Ellis Horowitz and Sartaj Sahni

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Objective

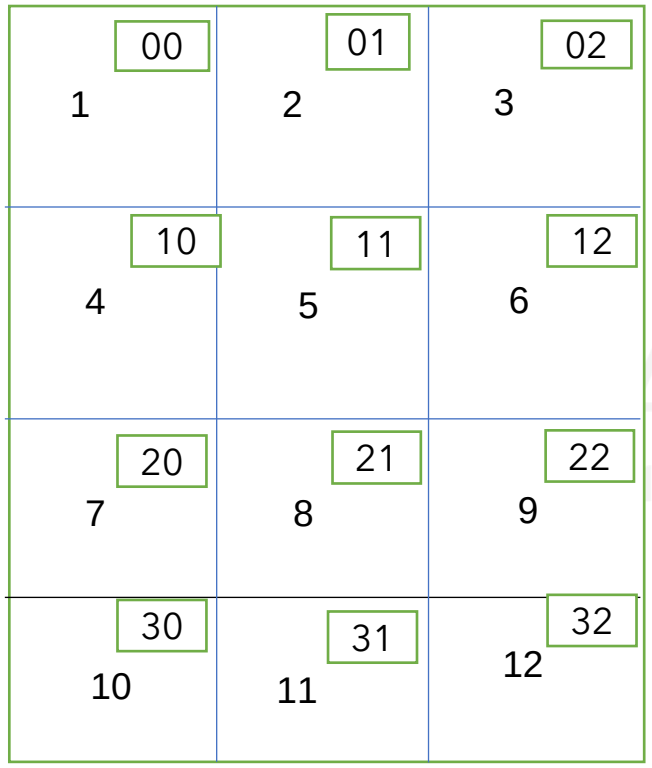
Two-Dimensional Array

2D Array representation

2D Arrays- Java

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

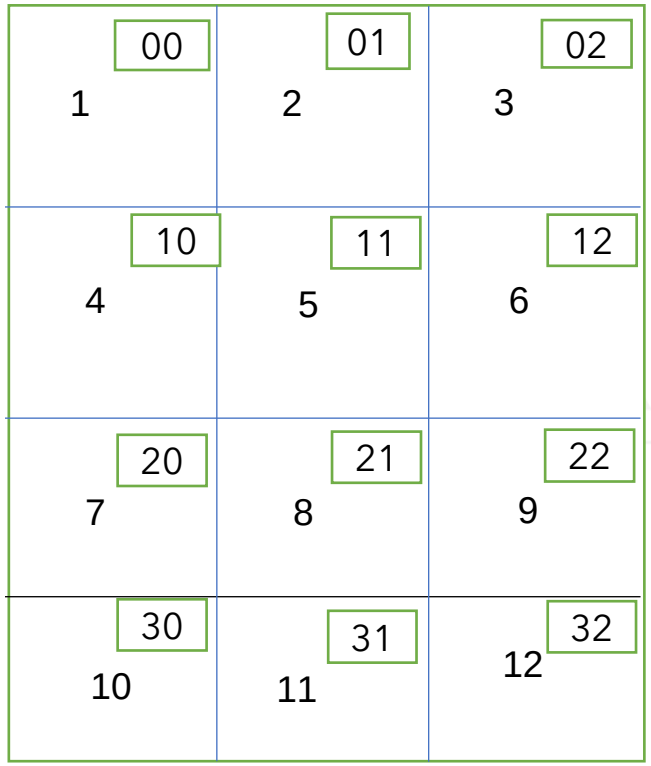
2-D Array



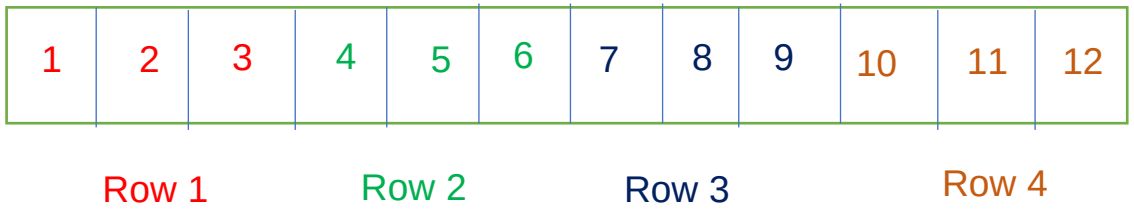
2D array with 4 rows and 3 columns.

- An element of 2 D array is accessed using two indices, row index and column index.
- If array name is A, 1st element is accessed using A[0][0].
- Last element is accessed using A[3][2].
- Element in 3rd row and 2nd column is accessed element is accessed as A[2][1].
- Element in i^{th} row and j^{th} column is accessed element is accessed as A[i-1][j-1]. (Assume array index starts with 0.)

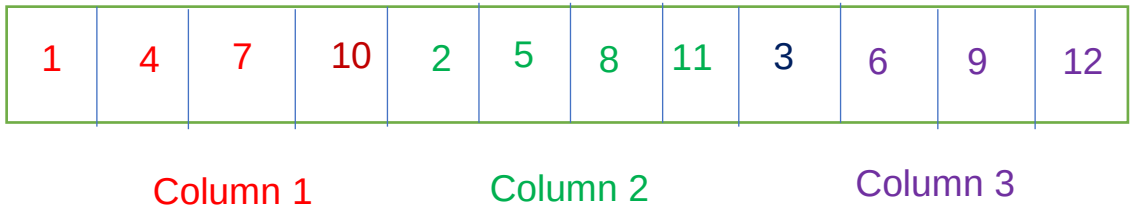
2-D Array Representation



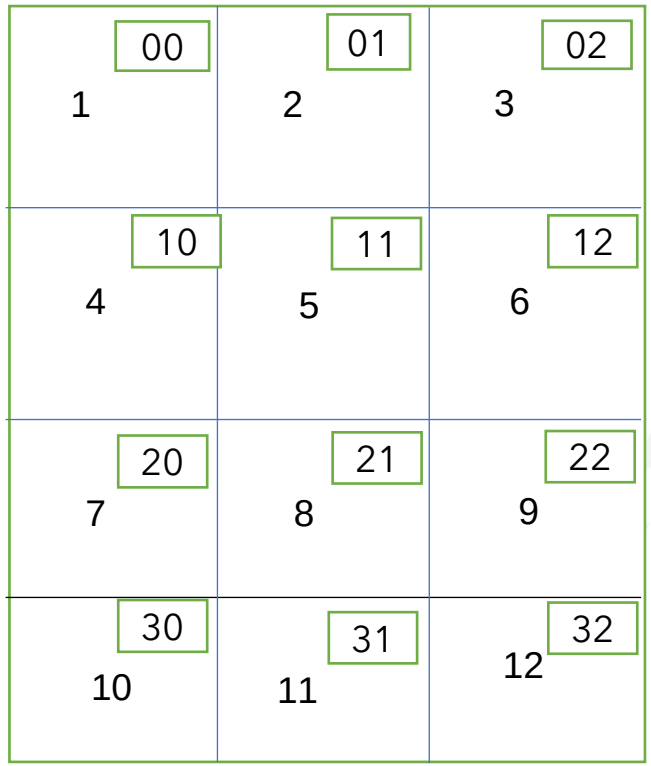
Row- major Representation



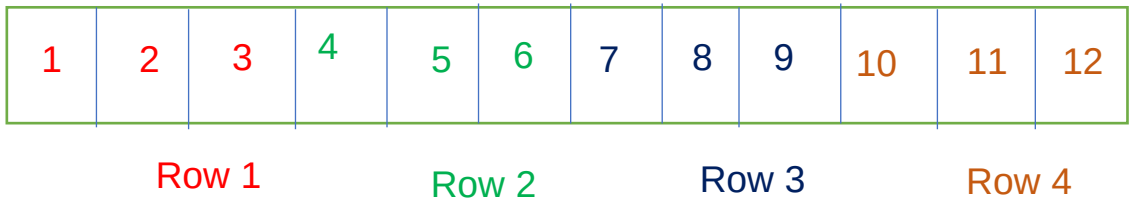
Column - major Representation



Row-major Representation



Row- major Representation

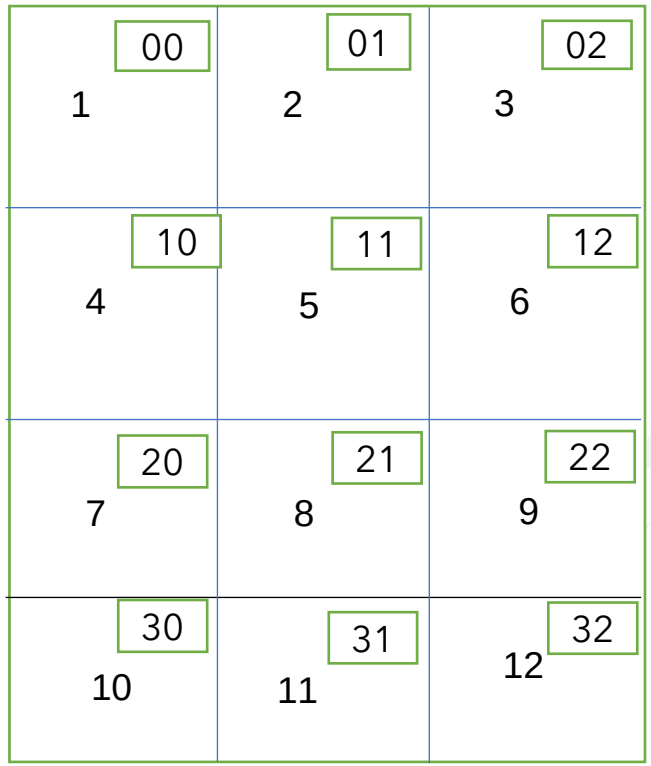


- Let Base Address =2000, 1 is stored in 2000, 2 is stored in 2004, 3 is stored in 2008 so on…..

(size of an integer is taken as 4 bytes.)

- Here no.of rows=4, no.of columns =3
- Address of A[2][1] =2000+4*(2*3+1) =2028
- Address of A[i][j] = base address+ size of an element * (rowindex*no.of columns+column index).

Column-major Representation



Column- major Representation

1	4	7	10	2	5	8	11	3	6	9	12

Column 1 Column 2 Column 3

- Let Base Address =2000, 1 is stored in 2000, 4 is stored in 2004, 7 is stored in 2008 so on…..
  (size of an integer is taken as 4 bytes.)
- Here No.of rows=4,No.of columns =3
- Address of A[2][1] =2000+4*(1*4+2) =2024
- Address of A[i][j] = base address+ size of an element * (colindex*no.of rows+row index).

2D Array Address Calculation - Example

Given a 2 D array, B with No.of rows = 4, No. of columns = 5 . Size of each item in the array is 1 byte. Base address of the array is 5000.

Find the address of the location B[4][3] in

Row-major Representation
Address of B[4][3]= 5000+1*(4*5+3) = 5023

Column-major Representation
Address of B[4][3]= 5000+1*(3*4+4)=5016

2D Array Representation

Row-major Representation

- C
- C++

Column-major Representation

- MathLab

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Two Dimensional Array - Java

- 2 D array – an array of arrays in Java.
- Doesn't follow row-major/column major representation.
- int arr[][] = new int[2][3]; //defines a 2D array with 2 rows and 3 columns.

Two Dimensional Array - Java

- Following snippet stores value into 2D array.

```
for(int i=0;i<arr.length;i++){
        for(int j=0;j<arr[i].length;j++){
                arr[i][j] = i+j;
        } }
```

- *arr.length* will return 2, the number of rows, *arr[0].length* returns number of elements(columns) in the zeroth row.

Two Dimensional Array - Java

Jagged array – array of arrays, where member arrays are of different sizes.

int jarray[][]=new int[2][]; //defines an array of arrays with size 2

jarray[0]= new int[3]; //defines first member array of size 3

jarray[1]=new int[5]; //defines second member array of size 5

Summary

- Two ways of array representations-row major, column major.
- Only Base address is stored in case of arrays.
- Java neither uses row major or column major representations.
- Java stores elements in contiguous locations in a simple array, but in array of arrays, each member array is stored in non contiguous locations

References

Data Structures & Problem Solving Using Java - mark allen weiss

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Objective

Stack

Stack ADT

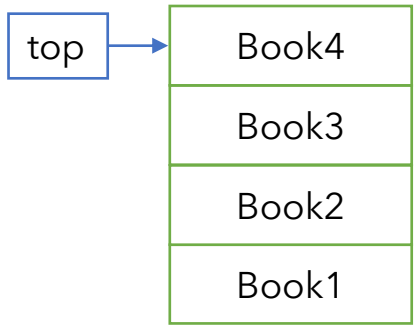
An application of stack

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

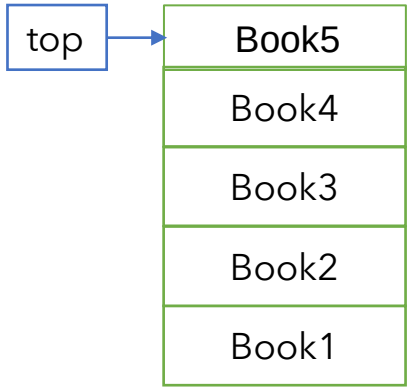
Stack

- Ordered List of items.
- Supports addition and removal of items only in one end, the top.
- Follows Last In First Out(LIFO) order in accessing items.

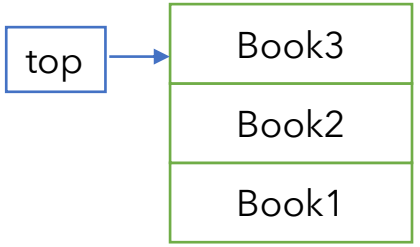
Stack of books



Add a book



Remove books



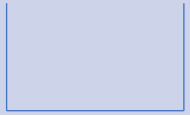
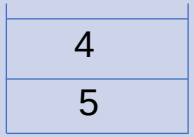
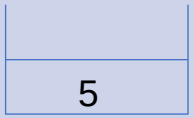
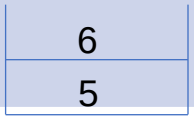
Stack as an ADT

Stack is a list of data items following LIFO order in data access.

Stack support the following operations:

- Push(item) → adds an item to the top.
- item pop() →removes an item from top.
- item peek()→return item at the top without removing it.
- Boolean isEmpty() → return true if Stack does not contain any elements.

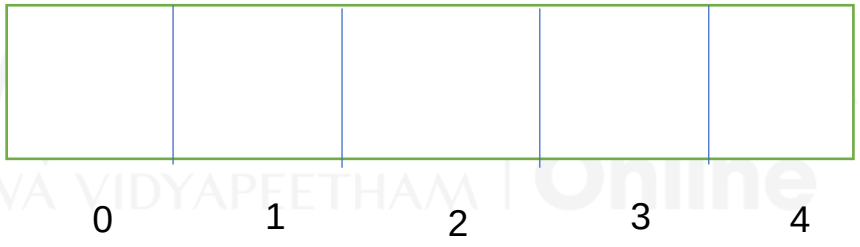
Stack – Logical Representation

Operation	Stack state	Return value
isEmpty()		True
Push(5)		Nothing
Push(4)		Nothing
isEmpty()		False
Pop()		4
Push(6)		Nothing
Peek()		6

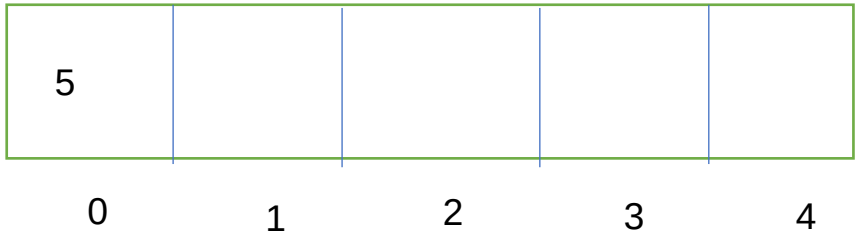
Stack ADT - Implementation

Stack can be implemented in a program using an **array/linked list.**

Given an array
int A[5];
int top=-1;



Push(5)
top++;
A[top]=5

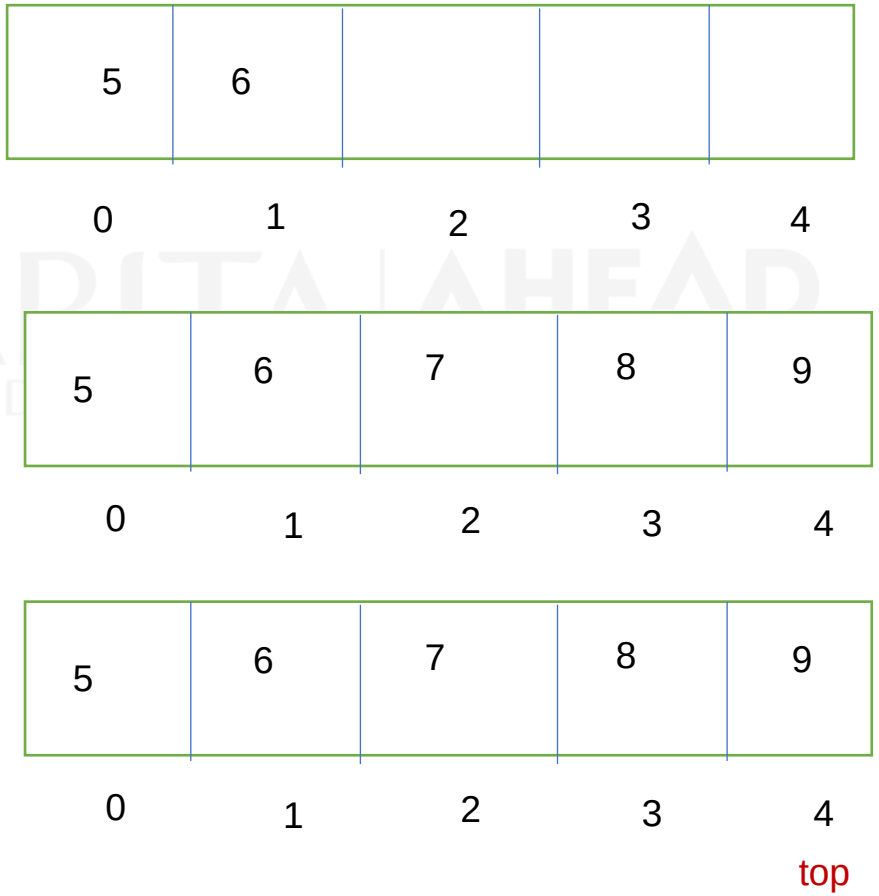


Stack ADT - Implementation

Push(6)
top++;
A[top]=6

After push(7),push(8)
Push(9), top=4

Push(10)

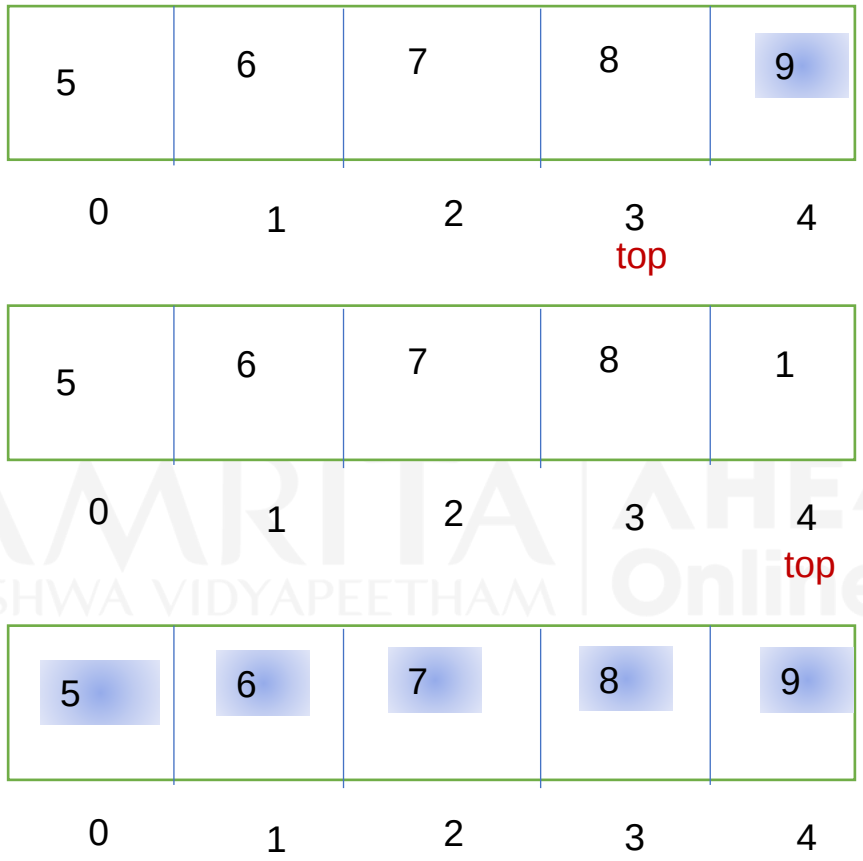


Stack ADT - Implementation

Pop()
top--

push(1)
top++
A[top]=1

Pop()
Pop()
Pop()
Pop()
Pop()
Pop()



Under flow

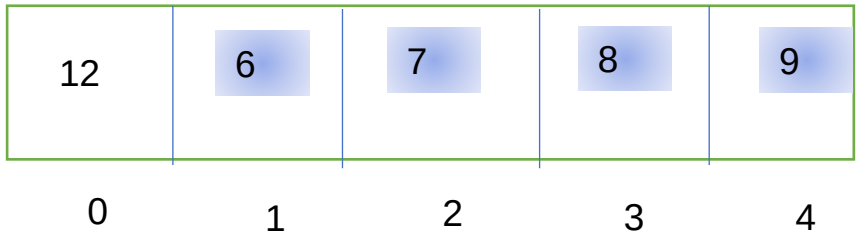
AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Stack ADT - Implementation

Push(12)

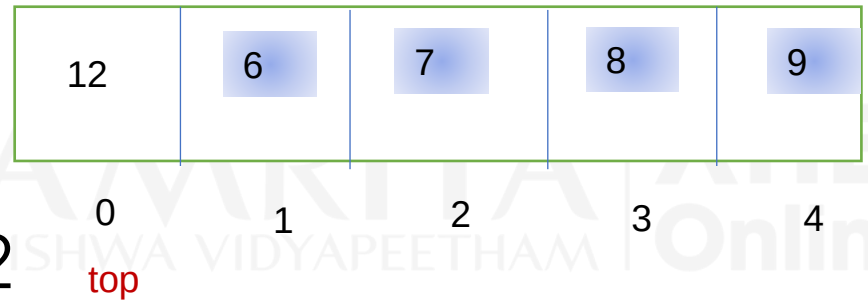
top++

A[top]=12



peek()

Returns the value 12



AMRITA VISHWA VIDYAPEETHAM AHEAD Online

Application of Stack

Given a parenthesised expression how will you check whether the parenthesis nesting is correct(Balanced expression) or not?

Balanced expression – {A+B * [C-D] } / (E+F)

Unbalanced Expression – (A * [B+D)]

Ans:- Use a stack, Push each opening parenthesis into the stack, and when a closing parenthesis is encountered, pop out and check for the corresponding opening parenthesis from stack.

Application of Stack : Example-1

Given (A+B) * (C-D)

Symbol	Operation	Stack Contents	Output
(	Push (	(	
A , + , B	Ignore	(	
)	Pop	Empty	
*	Ignore	Empty	
(	Push (	(	
C, -, D	Ignore	(	
)	Pop	Empty	True

Application of Stack : Example-2

Given ((A+B) * (C-D)

Symbol	Operation	Stack Contents	Output
(	Push (	(	
(	Push (	((	
A , + , B	Ignore	((	
)	Pop	(	
*	Ignore	(	
(	Push (	((	
C, -, D	Ignore	((	
)	Pop	(	
End of Exprn		Stack not Empty	False

Application of Stack : Example-3

Given (A+B) * (C-D))

Symbol	Operation	Stack Contents	Output
(	Push (	(	
A , + , B	Ignore	(	
)	Pop	Empty	
*	Ignore	Empty	
(	Push (	(	
C, -, D	Ignore	(	
)	Pop	Empty	
)	Pop	Empty	False

Application of Stack : Example-4

Given ([A+[B)

Symbol	Operation	Stack Contents	Output
(	Push (	(	
[	Push [	([	
A , +	Ignore	([	
[	Push [	([[	
B	Ignore	([[	
)	Pop Popped '[' and ')' doesn't match	([	False

Summary

LIFO property of stack

Stack and its Operations

Application of Stack Data Structure

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

References

Fundamentals of Data Structures - Ellis Horowitz and Sartaj Sahni

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

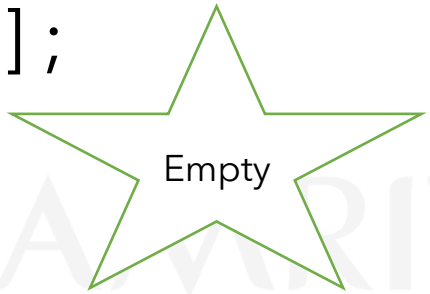
Objective

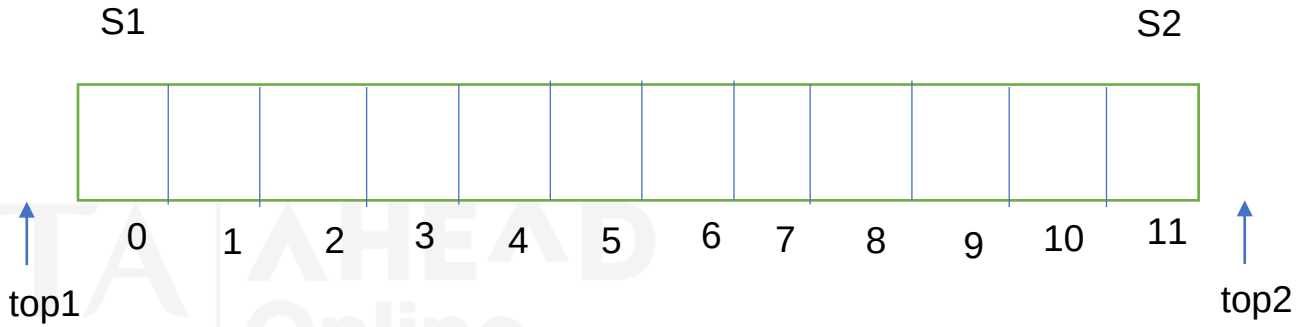
Two stacks in an array

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Two stacks in a single array

Given an array,
int A[ ] = new int[12] ;
int top1=-1;
int top2=12;





PushS1(i)
++top1;
A[top1]=i

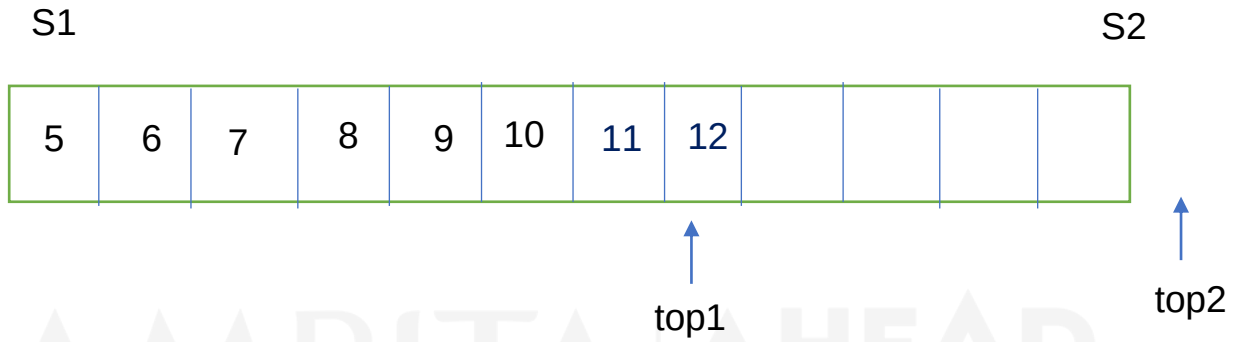
PushS2(j)
--top2;
A[top2]=j

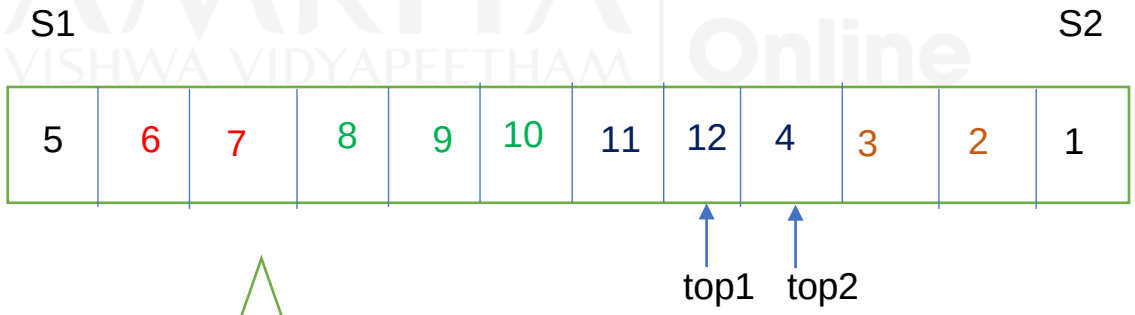
PopS1()
item=A[top1]
--top1;

PopS2()
Item=A[top2]
++top2;

Two stacks in a single array contd..

```
j=5
While(j<13)
  PushS1(j)
  j++

j=1
While(j<5)
  PushS2(j)
  j++

PushS2(7)
```

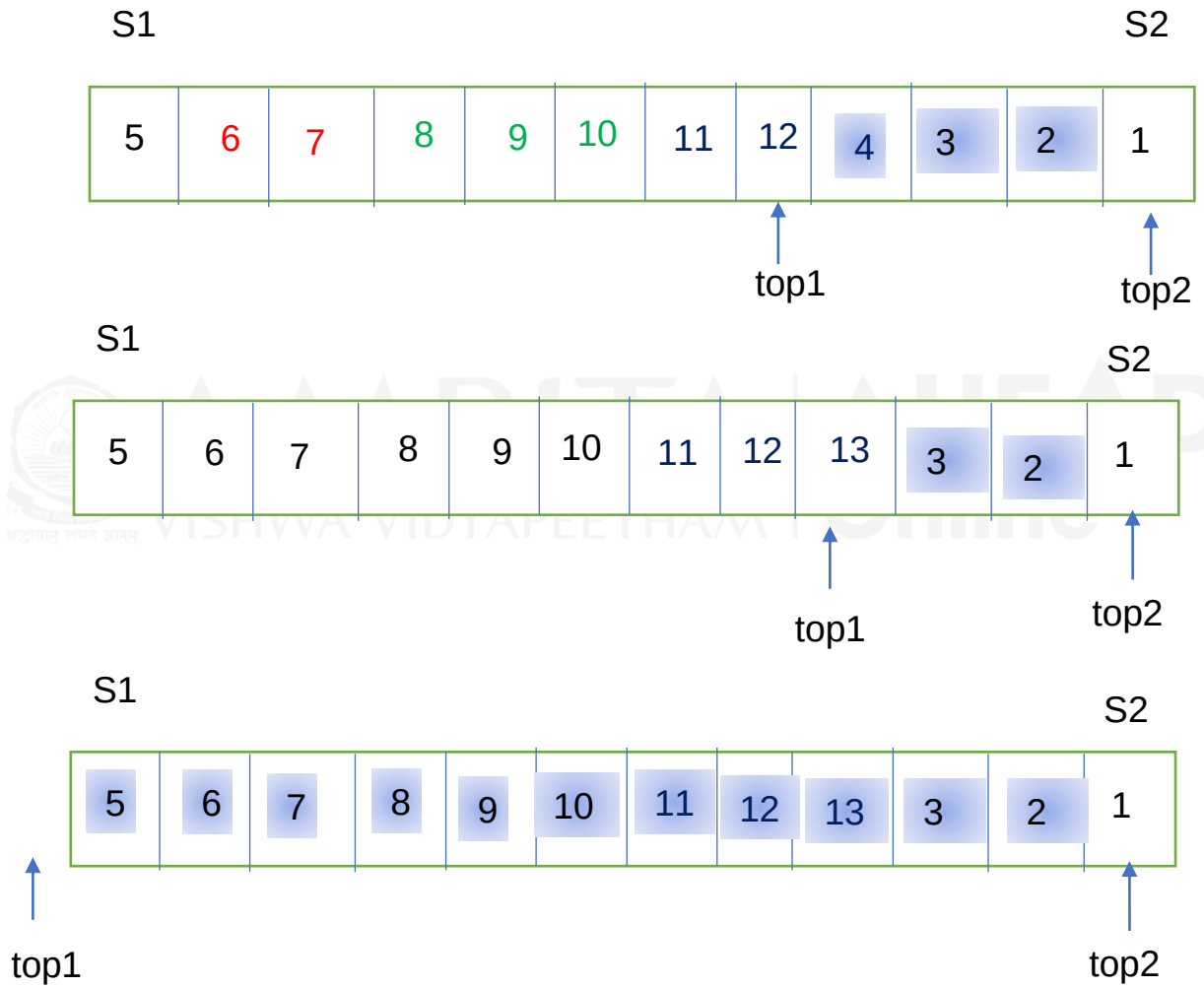




Now top1+1==top2

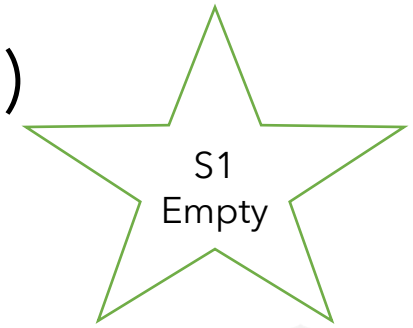
Two stacks in a single array contd..

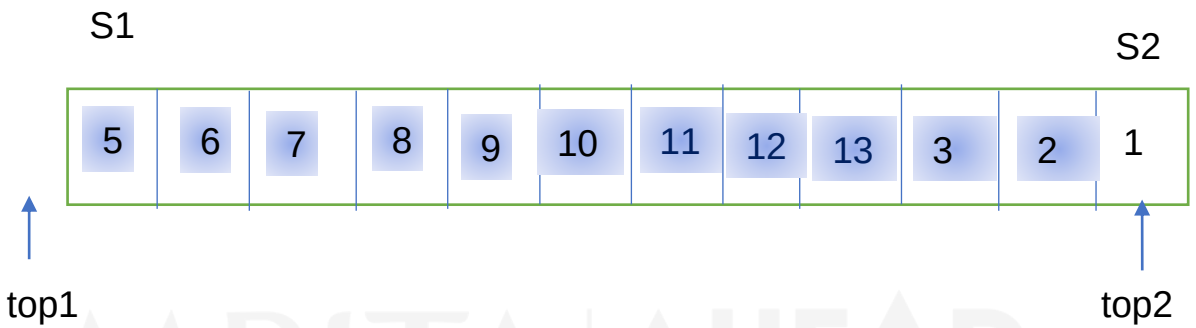
```
i=0
while(i<3)
  PopS2()
  i++

  PushS1(13)

  i=0
  while(i<10)
   PopS1()
```



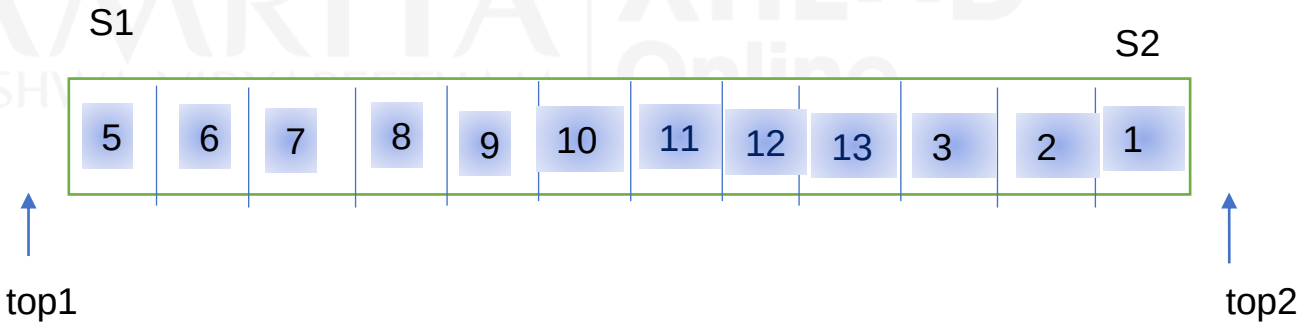
Two stacks in a single array contd..

PopS1()





PopS2()



PopS2()

Summary

Two stacks can be maintained in an array, by keeping stack1 in one end of the array and stack2 in other end of array.

Stacks become full when top1+1 = top2.

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

References

Fundamentals of Data Structures by Ellis Horowitz and Sartaj Sahni

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Objective

Multiple stacks in an array

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

‘p’ stacks in an array of size ‘q’

Given an array of size q, implement ‘p’ stacks.

Size of each stack= q/p

Following additional information to be maintained.

Top[p] → keep track of top of each stack.

BegStack[p]→keep track of beginning index of each stack.

EndStack[p]→keep track of last index of each stack.

Example- Three stacks in an array of size 14

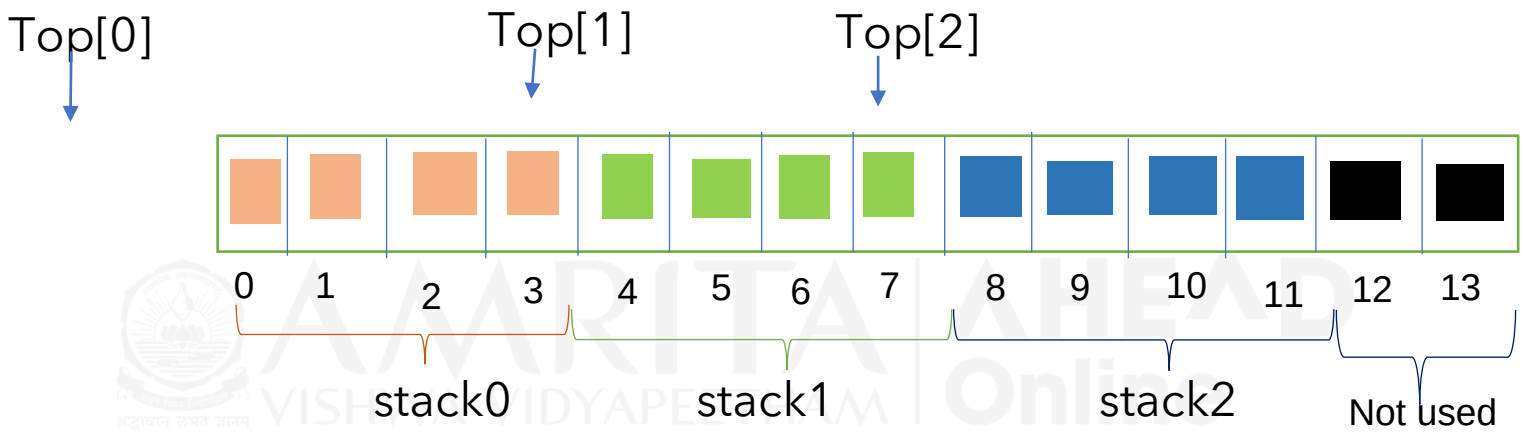
int Top[]= new int[3]; // three top variables,Top[0],Top[1],Top[2], one for each stack.

int BegStack[] =new int[3]; //to keep track of beginning of each stack.

int EndStack[] = new int[3]; //to keep track of end of each stack.

3 Stacks in an array of size 14

Given an array, int arr[]=new int[14]
Size of each Stack, Stksize = 14/3 = 4



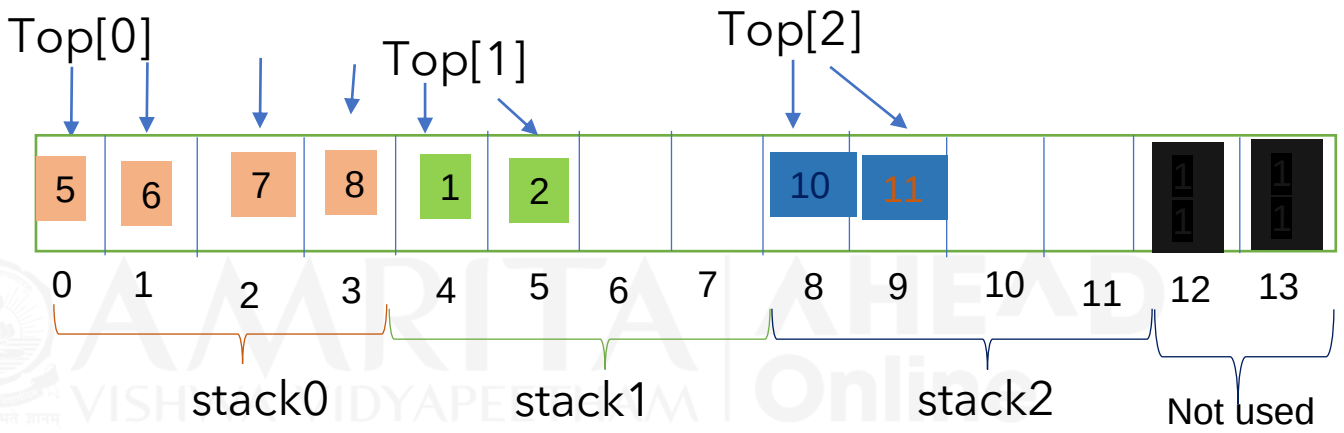
Top[0] =-1
BegStack[0] = 0
EndStack[0]= 3 (Stksize-1)

Top[1] = 3 (EndStack[0])
BegStack[1] =0+4=4
(BegStack[0]+Stksize)
EndStack[1]=3+4=7 (EndStack[0]+Stksize)

Top[2] = 7(EndStack[1])
BegStack[2] = 4+4 =8(BegStack[1]
+Stksize)
EndStack[2]= 11(EndStack[2] +Stksize)

3 stacks in an array of size 14

Given an array, int arr[]=new int[14]



Push(5,0)
Push(6,0)
Push(7,0)
push(8,0)
Push(9,0)

over flow

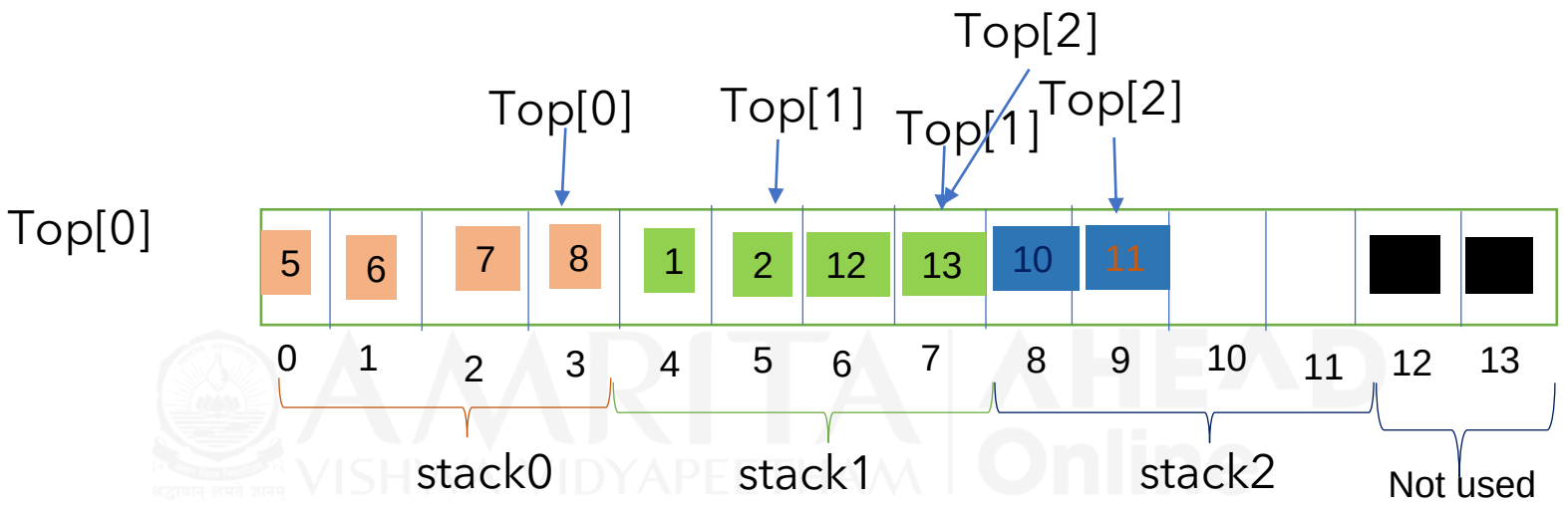
Push(1,1)
Push(2,1)
Push(10,2)
push(11,2)

3 stacks in an array of size 14

```
push(int item,int stacknum)  // 2 args, item and stacknumber
{
      if (stacknum>StkCount){ //StkCount is number of stacks in the array

                System.out.println("Invalid stack number" +Stk_num+" \n");
      }
  //check for a full stack
      else if (Top[stacknum]==EndStack[stacknum])
                print("stack "+stacknum +" is full");
       else{
                 ++Top[stacknum];
                arr[Top[stacknum]]=item;       }  }
```

3 stacks in an array of size 14

Given an array, int arr[]=new int[14]



Pop(0)
Pop(0)
Pop(0)
Pop(0)
Pop(0)

Under flow

Push(12,1)
Push(13,1)

Pop(2)
Pop(2)

Top[2]==EndStack[1]

Stack 2 Empty

3 stacks in an array of size 14

```
Pop(int stacknum){
{
      //Write code to check whether stacknum is valid

      // Check whether stack is empty
      if (Top[stacknum]==Endstack[stacknum-1])
            print("Empty Stack")
      else{
            int item=arr[Top[stacknum]];
            Top[stacknum]--;
            print( item);
      }
}
```

Reference

Data Structures and Algorithms made Easy – Narasimha Karumanchi

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online